B٠۶۶A"٠۶۶"٩F٠۶"٨F٠۶"٧F٠۶"۶F٠۶"۵F٠۶"۴F٠۶"٣F٠۶"٢F٠۶"١F٠۶"٠F٠۶"

دانشگاه صنعتی امیرکبیر

(پلی‌تکنیک تهران)

دانشکده مهندسی کامپیوتر

پروژه کارشناسی

پیاده‌سازی سیستم نگهداری و تعمیرات پیش‌بینانه تجهیزات بر بستر اینترنت اشیاء مبتنی بر تحلیل لرزش

نگارنده

استاد راهنما

دکتر حمیدرضا زرندی

تیر ۱۴۰۲


بسم الله الرحمن الرحيم

صفحه فرم ارزیابی و تصویب پایان نامه- فرم تأیید اعضاء کمیته دفاع

در این صفحه فرم دفاع یا تایید و تصویب پایان نامه موسوم به فرم کمیته دفاع- موجود در پرونده آموزشی- را قرار دهید.

نکات مهم:

- نگارش پایان نامه/رساله باید به زبان فارسی و بر اساس آخرین نسخه دستورالعمل و راهنمای تدوین پایان نامه های دانشگاه صنعتی امیرکبیر باشد.(دستورالعمل و راهنمای حاضر)
- رنگ جلد پایان نامه/رساله چاپی کارشناسی، کارشناسی ارشد و دکترا باید به ترتیب مشکی، طوسی و سفید رنگ باشد.
- چاپ و صحافی پایان نامه/رساله بصورت پشت و رو(دورو) بلامانع است و انجام آن توصیه می شود.

دانشگاه صنعتی امیرکبیر
(پلی‌تکنیک تهران)

به نام خدا

تعهدنامه اصالت اثر

تاریخ: تیر ۱۴۰۲

اینجانب متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

امضا

نویسنده پایان نامه، در صورت تمایل میتواند برای سپاسگزاری پایان نامه خود را به شخص یا اشخاص و یا ارگان خاصی تقدیم نماید.

سپاس‌گزاری

نویسنده پایان‌نامه می‌تواند مراتب امتنان خود را نسبت به استاد راهنما و استاد مشاور و یا دیگر افرادی که طی انجام پایان‌نامه به نحوی او را یاری و یا با او همکاری نموده‌اند ابراز دارد.

تیر۱۴۰۲

چکیده

در سال‌های اخیر اینترنت اشیاء به یکی از داغ‌ترین موضوعات فناوری تبدیل شده‌است. کاربرد این فناوری در تمامی حوزه‌های زندگی انسان و همچنین پیشرفت‌های اخیر در حوزه‌های جمع‌آوری داده، شبکه و هوش مصنوعی باعث شده‌اند که اینترنت اشیاء مورد توجه محققان قرار گیرد. یکی از چالش‌های موجود در صنایع و کارخانه‌ها تعویض بهینه قطعات است. با توجه به نبود اطلاعات کافی برای تحلیل وضعیت قطعات، راه‌حل مناسب برای اطمینان از کارکرد خط تولید، استفاده از متخصصان جهت بازرسی از وضعیت تجهیزات یا تعویض آنها بدون توجه به وضعیت فعلی و صرفا طبق زمان‌بندی قبلی است. این راه‌حل‌ها علاوه بر نداشتن دقت لازم، هزینه و زمان زیادی بر صنایع تحمیل می‌کنند. در این پروژه قصد داریم که با استفاده از ترکیب اینترنت اشیاء و هوش مصنوعی، چارچوبی برای تحلیل داده‌های لرزش تجهیزات ارائه گردد تا با استفاده از آن بتوان عمر مفید باقیمانده قطعات را بخوبی پیش‌بینی کرد. همچنین با استفاده از درگاه مبتنی بر وب، نتایج حاصله به افراد متخصص نشان داده شود تا بتوانند برنامه‌ریزی دقیقی برای نگهداری و تعمیر تجهیزات ارائه کنند. با بکارگیری این رویکرد هزینه و وقت صرف‌شده برای تعویض قطعات به‌شکل چشمگیری کاهش می‌یابد.

واژه‌های کلیدی:

اینترنت اشیاء، نگهداری پیش‌بینانه، اینترنت اشیاء صنعتی، نگهداری سیستم سایبری-فیزیکی

فهرست مطالب

عنوان صفحه

۱ **مقدمه** ۱

۱-۱ تعمیر و نگهداری ۲

۱-۲ راهکار پیشنهادی ۳

۱-۳ کارهای مشابه ۳

۲ **مفاهیم پایه و تجهیزات** ۵

۲-۱ حسگر ۶

۲-۲ گره انتهایی ۷

۲-۳ پروتکل ارتباطی ۸

۲-۳-۱ استاندارد زیگبی ۸

۲-۳-۲ ویژگی‌های زیگبی ۹

۲-۳-۳ دسته‌بندی دستگاه‌ها ۱۱

۲-۳-۴ توپولوژی ۱۲

۲-۳-۵ دیجی بین‌الملل ۱۳

۲-۳-۶ طرز کار دستگاه‌های ایکس‌بی ۱۳

۲-۳-۷ ارتباط سریال ماژول‌های ایکس‌بی ۱۵

۲-۴ دروازه ۱۷

۲-۵ پایگاه داده‌ی سری زمانی ۱۸

۲-۶ پیش‌پردازش ۱۹

۲-۷ برنامه‌ی تحت وب ۲۰

۲-۸ جمع‌بندی و نتیجه‌گیری ۲۱

۳ **پیاده‌سازی سیستم** ۲۲

۳-۱ معماری کلی سیستم ۲۳

۳-۲ گره انتهایی و آردوینو ۲۳

۳-۲-۱ خواندن از حسگر و نمونه‌برداری ۲۴

۳-۲-۲ تشکیل فریم زیگبی و نوشتن در درگاه سریال ۲۵
۳-۲-۳ خوابیدن و بیدارشدن در بازه‌های زمانی مشخص ۲۸
۳-۳ دروازه ۲۹
۳-۳-۱ فایل env_vars.py ۲۹
۳-۳-۲ فایل models.py ۲۹
۳-۳-۳ فایل receiver.py ۳۰
۳-۳-۴ فایل requirements.txt ۳۰
۳-۳-۵ فایل schedule.sh ۳۰
۳-۳-۶ فایل send_data.py ۳۱
۳-۳-۷ فایل signup.py ۳۱
۳-۳-۸ فایل utils.py ۳۱
۳-۴ پیش‌پردازش ۳۱
۳-۵ پایگاه داده سری زمانی ۳۲
۳-۵-۱ ساختار کلی ۳۳
۳-۵-۲ اندازه‌گیری‌ها و توابع مربوط به نوشتن و گرفتن داده ۳۳
۳-۵-۳ توابع گرفتن شناسه گره و اندازه‌گیری ۳۴
۳-۵-۴ تابع get_starting_service_date ۳۴
۳-۵-۵ تابع حذف داده‌ها و حافظه پنهان ۳۴

منابع و مراجع ۳۵

پیوست ۳۸

واژه‌نامه‌ی فارسی به انگلیسی ۳۹

واژه‌نامه‌ی انگلیسی به فارسی ۴۱

فهرست اشکال

شکل صفحه

۱-۱ سیاست‌های انجام تعمیرات و نگهداری [۱] ۲

۲-۱ حسگر مدل ADXL 345 . . . . . ۷

۲-۲ بورد آردوینو نانو ۷

۲-۳ مقایسه پروتکل زیگبی و سایر پروتکل‌های اینترنت اشیاء ۹

۲-۴ شبکه زیگبی شامل هماهنگ‌کننده، مسیریاب و گره‌پایانی [۱۰] ۱۲

۲-۵ توپولوژی‌های مختلف شبکه زیگبی [۱۱] ۱۳

۲-۶ ماژول زیگبی مدل XBee S2 . . . . . ۱۴

۲-۷ فرایندهای لازم برای ارتباط دو ماژول ایکس‌بی [۱۲] ۱۴

۲-۸ ارتباط سریال در حالت شفاف [۱۲] ۱۵

۲-۹ ارتباط سریال در حالت API [۱۲] ۱۷

۲-۱۰ بورد زبری‌پای ۴ مدل بی ۱۸

۲-۱۱ پلتفرم متن باز پایگاه داده سری زمانی InfluxDB [۱۵] ۱۹

۳-۱ نمای کلی سیستم ۲۴

۳-۲ ساختار فایل کتابخانه ایکس‌بی ۲۶

۳-۳ ساختار کلی فریم API [۱۲] ۲۶

۳-۴ ساختار پروژه دروازه ۲۹

۳-۵ ساختار کلی پایگاه داده سری زمانی ۳۳

۳-۶ کد پرس‌وجوی vibration_measurement ۳۴

فهرست جداول

جدول صفحه

۱-۲ مقایسه بین دو نوع حسگر مبتنی بر پیزوالکتریک و MEMS [۵] ۶

۱-۳ بخش‌های مختلف داده‌های فریم درخواست ارسال [۱۲] ۲۷

فهرست نمادها

نماد	مفهوم
$\mathbb{R}^n$	فضای اقلیدسی با بعد n
$\mathbb{S}^n$	کره یکه n بعدی
M^m	خمینه m-بعدی M
$\mathfrak{X}(M)$	جبر میدان‌های برداری هموار روی M
$\mathfrak{X}^{\backprime}(M)$	مجموعه میدان‌های برداری هموار یکه روی (M,g)
$\Omega^p(M)$	مجموعه p-فرمی‌های روی خمینه M
Q	اپراتور ریچی
$\mathcal{R}$	تانسور انحنای ریمان
ric	تانسور ریچی
L	مشتق لی
Φ	۲-فرم اساسی خمینه تماسی
∇	التصاق لوی-چویتای
Δ	لاپلاسین ناهموار
∇^*	عملگر خودالحاق صوری القا شده از التصاق لوی-چویتای
g_s	متر ساساکی
∇	التصاق لوی-چویتای وابسته به متر ساساکی
Δ	عملگر لاپلاس-بلترامی روی p-فرم‌ها

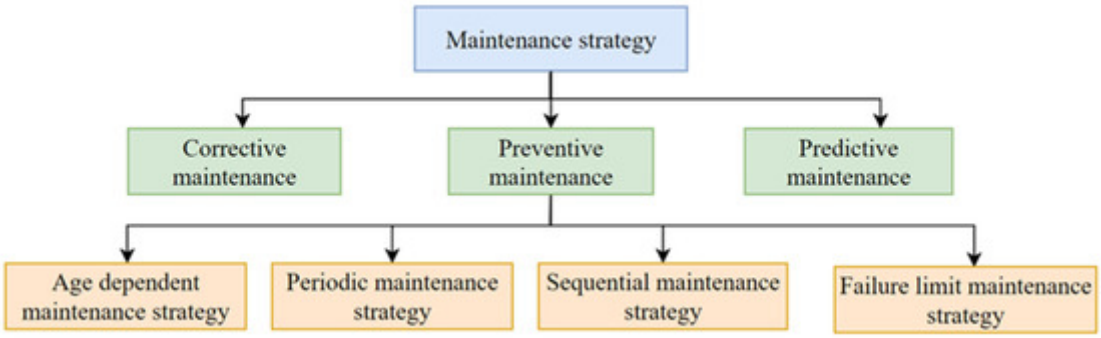
فصل اول

مقدمه

۱-۱ تعمیر و نگهداری

در دنیای نوین امروز، وابستگی به فناوری و صنایع مختلف برای ارائه خدمات و تولید محصولات مورد نیاز بشدت افزایش یافته‌است. در نتیجه این افزایش، تعداد کارخانه‌ها و تجهیزات برای رفع این نیازها نیز افزایش یافته‌است. همه تجهیزات و وسایل پس از استفاده زیاد دچار نقص و فراوانی می‌شوند. خرابی تجهیزات در کارخانه‌ها و حطوط تولید پدیده‌ای نامطلوب است. خرابی تجهیزات باعث ایست خط تولید و وارد کردن ضرر مالی به صاحبان کارخانه‌ها خواهد شد. به‌همین دلیل، تعمیر و نگهداری تجهیزات اهمیت بالایی دارد.

سیاست‌های انجام تعمیرات و نگهداری به سه دسته اصلی تقسیم می‌شوند؛ که در شکل ۱-۱ [۱] آورده شده‌است:



شکل ۱-۱: سیاست‌های انجام تعمیرات و نگهداری [۱]

این سیاست‌ها عبارتند از:

- نگهداری اصلاحی[1]: این نوع نگهداری با اتفاق افتادن خرابی رخ می‌دهد و تنها در صورت خرابی قطعه تعمیر انجام می‌شود. همانطور که مشخص است در این حالت روند کارکرد عادی متوقف می‌شود: به‌علاوه، در این حالت تعمیر می‌تواند با تاخیر انجام شود زیرا ممکن است در لحظه قطعات مورد نیاز برای تعمیر موجود نباشد. از این سیاست بیشتر به‌عنوان مکملی برای سیاست‌های بعدی استفاده می‌شود[۱].

- نگهداری پیش‌گیرانه[2]: در این حالت، نگهداری و تعمیر بر اساس رابطه بین نرخ خرابی، توزیع زمان خرابی و سایر آستانه‌هایی که از تعداد زیادی از آمارهای خرابی بدست می‌آید زمان‌بندی می‌شود. در واقع در این نوع نگهداری هدف کاهش احتمال خرابی قطعات است. این نوع نگهداری

[1] Corrective Maintenance
[2] Preventive Maintenance

با توجه به نوع اطلاعات به سیاست‌های وابسته عمر،[۳] سیاست‌های دوره‌ای[۴]، سیاست‌های ترتیبی[۵] و سیاست‌های محدودکننده خرابی[۶] تقسیم می‌شوند[۱].

- نگهداری پیش‌بینانه[۷]: این نوع نگهداری بر روند کاهش کارایی قطعات با استفاده از وضعیت آنها نظارت کرده، وضعیت آنها را در آینده پیش‌بینی می‌کند و مداوم برنامه تعمیر و نگهداری را بروزرسانی می‌کند، تا اینکه شرایط توقف بروزرسانی برآورده شود. با توجه به این ویژگی، این سیاست فقط برای تجهیزاتی که وضعیت آنها قابل نظارت است می‌تواند استفاده شود[۱].

۱-۲ راهکار پیشنهادی

با توجه به اهمیت نگهداری و نقش نگهداری اصولی در کاهش هزینه، در این پروژه به ایجاد چارچوبی مناسب برای ارائه راهکاری بر مبنای سیاست نگهداری پیش‌بینانه می‌پردازیم. یکی از نیازهای این سیاست حجم زیادی از داده جمع‌آوری شده از وضعیت تجهیزات برای پیش‌بینی دقیق است. در این پروژه در بستر اینترنت اشیاء[۸] داده‌های لرزش با استفاده از سنسور MEMS[۹] جمع‌آوری شده و در زمان‌های مشخص با استفاده از پروتکل زیگبی[۱۰] برای دروازه[۱۱] فرستاده می‌شود. پس از جمع‌آوری حجم مشخصی از داده‌هااطلاعات برای پردازش بیشتر به سرور فرستاده می‌شوند. در انتها نیز اطلاعات پردازش‌شده و تخمین عمر مفید باقیمانده[۱۲] در یک صفحه وب نمایش داده می‌شود.

۱-۳ کارهای مشابه

در [۲] مدلی برای خرابی قطعات مبتنی بر میزان استفاده از تجهیزات و بار داخلی آنها ارائه شده‌است اما مدل ارائه‌شده محدود به یک مدل تجهیزات است و برای استفاده در سایر مدل‌ها نظارت مجدد لازم است. در [۳] رویکردی مبتنی بر شبکه عصبی جهت پیش‌بینی عمر مفید باقیمانده تجهیزات چرخشی

[3] Age Dependent Strategies
[4] Periodic Strategies
[5] Sequential Strategies
[6] Failure Limited Strategies
[7] Predictive Maintenance
[8] Internet of Things
[9] Micro Electric Mechanical Sensor
[10] Zigbee Protocol
[11] Gateway
[12] Remaining Useful Lifetime(RUL)

ارائه شده‌است اما تنها مختص به این دسته از تجهیزات است. در [۴] رویکردی مبتنی بر شبکه عصبی جهت پیش‌بینی زمان نگهداری و تعمیرات تجهیزات بر اساس مدل خرابی و کارایی آنها ارائه می‌شود اما در یک محیط شبیه‌سازی‌شده و کنترل‌شده آزمایش شده‌است و در هنگام استفاده در محیط واقعی غیرعملی است.

بطور کلی در پژوهش‌های یادشده رویکردهای ارائه‌شده مختص نوع خاصی از تجهیزات است و در یک مجیط آزمایشگاهی و کنترل‌شده ارزیابی شده‌اند. در حالیکه در این پروژه با استفاده از داده‌های جمع‌آوری‌شده از قطعات مختلف سعی کرده‌ایم رویکردی کلی و مناسب محیط واقعی و صنعتی ارائه دهیم.

فصل دوم

مفاهیم پایه و تجهیزات

در این فصل به بررسی مفاهیم و تجهیزاتی که در این پروژه استفاده شده‌است می‌پردازیم. در هر بخش دلیل انتخاب خود را شرح می‌دهیم و آن را با سایر راه‌حل‌ها مقایسه می‌کنیم.

۲-۱ حسگر

همانطور که گفته‌شد برای پیش‌بینی نگهداری و عمر مفید تجهیزات نیاز به نظارت و جمع‌آوری اطلاعاتی راجع‌به این تجهیزات داریم. در این پروژه، اطلاعات جمع‌آوری‌شده مربوط به وضعیت لرزش تجهیزات است که با استفاده از حسگرهای لرزش MEMS اندازه‌گیری می‌شوند.

در این پروژه تصمیم گرفتیم که بجای دما یا رطوبت از لرزش تجهیزات برای پیش‌بینی استفاده کنیم؛ زیرا لرزش برخلاف دو معیار دیگر بطور مستقیم شرایط عملیاتی تجهیزات را منعکس می‌کند که بیشتر بر اساس رفتارهای مکانیکی مانند چرخش موتور یا حرکت جریان در لوله ایجاد می‌شوند. هم‌چنین دو معیار دیگر وابستگی زیادی به محیط دارند اما لرزش تقریبا مستقل از عوامل خارجی است[۵].

برای اندازه‌گیری لرزش دو نوع حسگر لرزش وجود دارد. حسگرهای مبتنی بر شتاب‌سنج MEMS بر اکثر محدودیت‌های حسگرهای قدیمی مبتنی بر شتاب‌سنج پیزوالکتریک[1] غلبه می‌کنند. تفاوت این دو حسگر را می‌توان در جدول ۲-۱ [۵] دید.

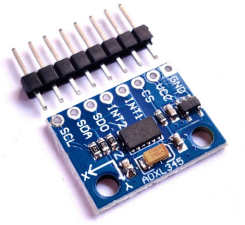
جدول ۲-۱: مقایسه بین دو نوع حسگر مبتنی بر پیزوالکتریک و MEMS [۵]

	پیزوالکتریک	MEMS
قیمت (دلار)	۳۰۰+	۱۰+
توان مصرفی (mW)	۲۷	۳
اندازه (اینچ)	۱ × ۰/۹۸ × ۱/۹۷	۰/۵ × ۰/۲ × ۰/۲
تراکم نویز (μg)	۷۰۰	۴۰۰۰
بازه شتاب (g)	۱۰	۱۰۰

بطور کلی حسگرهای نوع MEMS ارزان‌تر، کم‌مصرف‌تر و کوچک‌تر هستند. در این پروژه حسگرها با باتری کار خواهند کرد و برای اندازه‌گیری به قطعات مورد نظر متصل می‌شوند. بنابراین کم‌مصرف‌بودن، ارزانی و کوچک‌بودن برای اتصال به بدنه ویژگی‌های مطلوبی است که در حسگرهای نوع MEMS یافت می‌شود. شکل ۲-۱ حسگر مدل ADXL 345 که در این پروژه استفاده کرده‌ایم.

این حسگر می‌تواند شتاب را در سه جهت، در چهار بازه قابل تنظیم ۲، ۴، ۸ و ۱۶ برابر گرانش با

[1] Piezoelectric

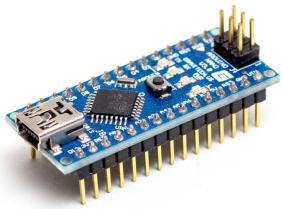


شکل ۲-۱: حسگر مدل ADXL 345

دقت‌های متفاوت اندازه‌گیری کند. خروجی این حسگر نیز با دو پروتکل SPI[۲] و I2C قابل انتقال است.

۲-۲ گره انتهایی

برای کاهش بیشتر مصرف انرژی، دریافت اطلاعات حسگر و فرستادن آن برای دروازه به یک بورد[۳] نیاز داریم. بوردهای آردوینو[۴] اگرچه محدودیت‌هایی دارند، اما با توجه به ارزان‌بودن و برآورده‌کردن نیاز ما انتخاب مناسبی هستند. بورد استفاده‌شده در این پروژه آردوینو نانو است که در شکل ۲-۲ قابل‌مشاهده است. در این بورد از پورت سریال[۵] برای ارتباط با ماژول زیگبی[۶] و از پروتکل I2C برای ارتباط با حسگر استفاده می‌کنیم. یکی از محدودیت‌های این بورد حافظه کم است که در نرخ‌های بالای نمونه‌برداری حسگر کار ما را سخت می‌کند که در فصل بعد بیشتر به آن می‌پردازیم.



شکل ۲-۲: بورد آردوینو نانو

[2] Serial Peripheral Interface
[3] Board
[4] Arduino
[5] Serial Port
[6] Zigbee Module

۲-۳ پروتکل ارتباطی

برای ارسال اطلاعات از گره‌های متصل به حسگر به دروازه نیاز به یک پروتکل ارتباطی داریم. نیازهای ما در رابطه با این پروتکل عبارتند از:

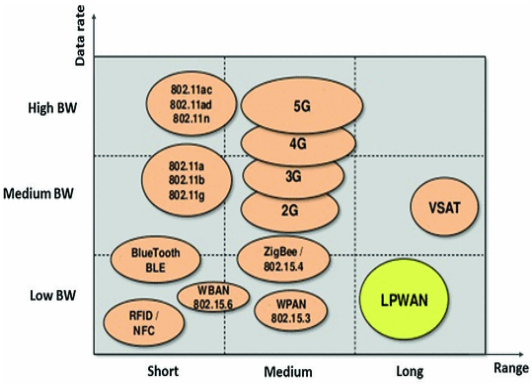
- کم‌مصرف‌بودن: با توجه به اینکه گره‌های ما به باتری متصل هستند، به پروتکلی نیاز داریم که در مصرف انرژی بسیار بهینه عمل کند تا بتوان بدون تعویض باتری اطلاعات را تا چند سال برای دروازه فرستاد.
- پوشش و نفوذ سیگنال: در هر کارخانه یک دروازه نصب شده‌است، کارخانه‌ها مساحتی متوسط دارند و مانع خاصی نیز در آنها وجود ندارد. پس نیاز به پوشش یا قدرت نفوذ بالایی در سیگنال ارسالی نداریم.
- توپولوژی[۷]: با توجه به وجود یک دروازه و چندین گره که همگی اطلاعات را برای دروازه ارسال می‌کنند، نیاز به پروتکلی داریم که از توپولوژی ستاره پشتیبانی کند.
- ارتباط مطمئن[۸]: ارسال ناقص و یا از دست‌دادن اطلاعات حسگرها می‌تواند باعث پیش‌بینی اشتباه شود. بنابراین، به پروتکلی با قابلیت ارتباط مطمئن و کنترل خطا نیاز داریم.

در شکل ۲-۳ پروتکل‌های ارتباطی با معیارهای مختلف مقایسه شده‌اند. بر اساس ویژگی‌های موردنیاز پروتکل زیگبی انتخاب مناسب‌تری نسبت به سایر پروتکل‌ها است که در ادامه آن را معرفی می‌کنیم.

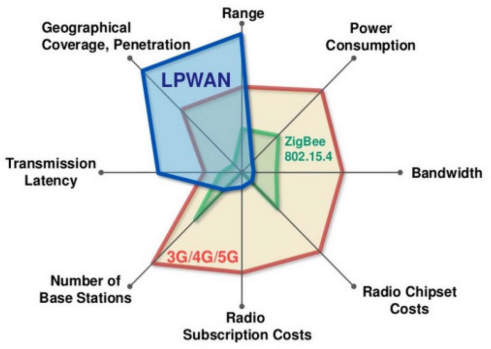
۲-۳-۱ استاندارد زیگبی

زیگبی یکی از استانداردهای فرستنده-گیرنده[۹] در شبکه‌های حسگر بی‌سیم[۱۰] است که بصورت گسترده مورد استفاده قرار گرفته‌است. زیگبی بر روی IEEE802.15.4 مشخصات شبکه شخصی بی‌سیم با نرخ داده پایین[۱۱] را تعریف می‌کند تا بتواند از دستگاه‌های نظارتی و کنترلی با توان مصرفی و نرخ داده پایین پشتیبانی کند. زیگبی توسط اتحاد زیگبی[۱۲] توسعه داده شده‌است؛ که صدها عضو دارد. اتحاد زیگبی

[7] Topology
[8] Reliable Communication
[9] Tranceiver
[10] Wireless Sensor Network
[11] Low Rate Wireless Personal Area Network(LR-WPAN
[12] Zigbee Alliance



(آ) مقایسه بر حسب نرخ داده و بازه پوشش [۶]



(ب) مقایسه بر حسب انرژی، بازه پوشش، نفوذ، تاخیر، هزینه و پهنای باند [۷]

شکل ۲-۳: مقایسه پروتکل زیگبی و سایر پروتکل‌های اینترنت اشیاء

لایه‌های شبکه، امنیت و برنامه و IEEE802.15.4 لایه‌های سخت‌افزار و کنترل دسترسی به رسانه[۱۳] را تعریف می‌کنند[۸].

استاندارد شبکه بی‌سیم زیگبی جای خالی‌ای را در بازار پر می‌کند که توسط سایر شرکت‌ها نادیده گرفته شده‌است. اکثر استانداردهای بی‌سیم به دنبال نزدیک‌شدن به اینترنت و رسیدن به سرعت‌های بالاتر هستند. اما زیگبی در تلاش است تا با ارائه نرخ ارسال پایین‌تر انتظار مصرف انرژی پایین را برآورده کند. سایر استانداردها به دنبال افزودن ویژگی[۱۴]های بیشتر و ارائه خدماتی نظیر استریم با کیفیت بالا[۱۵] هستند؛ درحالیکه زیگبی با پشته[۱۶]ای کوچک تنها هدف دارد که بتوان داده‌هایی با حجم کم و تعدد پایین مانند کنترل چراغ یا خواندن داده‌های سنسور دما را ارسال کرد[۹].

۲-۳-۲ ویژگی‌های زیگبی

قابلیت اطمینان بالا

ارتباطات بی‌سیم بطور کلی ارتباطات نامطمئنی هستند اما زیگبی در چند سطح قابلیت اطمینان بالایی فراهم می کند که عبارتند از:

[13] Media Access Control(MAC)
[14] Feature
[15] High-Definition Streaming
[16] Stack

- IEEE802.15.4: در آن از ترکیبی از فناوری‌هایی مانند O-QPSK[17] و DSSS[18] استفاده می‌شود که کارایی بسیار خوبی در محیط‌های با نرخ سیگنال به نویز پایین دارند[۹].

- CSMA-CA[19]: زیگبی از این فناوری برای کنترل دسترسی به رسانه استفاده می‌کند. قبل از ارسال زیگبی به کانال گوش می‌دهد. اگر کسی در حال ارسال نباشد، اطلاعات خود را ارسال می‌کند. این روند از تداخل اطلاعات فرستنده‌های مختلف و ایجاد نیاز برای ارسال دوباره جلوگیری می‌کند[۹].

- کنترل خطا: در هر فریم[20] زیگبی از CRC[21] ۱۶ بیتی بعنوان چک‌سام[22] استفاده می‌شود که قابلیت تشخیص خطا در هر فریم را ایجاد می‌کند[۹].

- فریم تایید: هر فریم در کل ۴ بار برای مقصد ارسال می‌شود(۳ بار ارسال مجدد). اگر باز هم فریم نتواند ارسال شود به مبدا اطلاع داده می‌شود[۹].

مصرف انرژی پایین

ماژول‌های زیگبی در مصرف انرژی بسیار بهینه هستند. یک شبکه زیگبی می‌تواند تا چند سال تنها با استفاده از باتری عمل کند. اگر استفاده از زیگبی به‌درستی مدیریت شود این زمان حتی می‌تواند به عمر باتری اگر بدون استفاده در قفسه بماند برسد. دلیل این استفاده کم این است که گره‌های زیگبی می‌توانند به خواب بروند. نیازی به نگه‌داشتن ارتباط برای باقی‌ماندن در شبکه ندارند[۹].

امنیت بالا

زیگبی با استفاده از استاندارد رمزگذاری پیشرفته[23] باعث می‌شود که تنها فرستنده و گیرنده از محتویات فریم اطلاع داشته باشند. هم‌چنین روندی برای احراز هویت گره‌ها هنگام اضافه‌شدن به شبکه بکار می‌برد که مانع اضافه‌شدن گره‌های مخرب به شبکه می‌شود[۹].

[17] Offset-Quadrature Phase-Shift Keying
[18] Direct Sequence Spread Spectrum
[19] Carrier Sense Multiple Access Collision Avoidance
[20] Frame
[21] Cyclic Redundancy Check
[22] Checksum
[23] Advanced Encryption Standard(AES)

۲-۳-۳ دسته‌بندی دستگاه‌ها

دسته‌بندی فیزیکی

با توجه به توانایی‌های پردازشی دو نوع دستگاه در IEEE802.15.4 آورده شده‌است:

- دستگاه با عملکرد کامل[۲۴]: این دستگاه‌ها توانایی انجام همه‌ی اعمال استاندارد مانند مسیریابی[۲۵]، هماهنگی[۲۶] و ارسال اطلاعات حسگر را دارند. در استاندارد فعلی این دستگاه‌ها باید همیشه روشن باشند[۸].
- دستگاه با عملکرد کاهش‌یافته[۲۷]: این دستگاه‌ها تنها توانایی ارسال داده‌های حسگر را دارند و می‌توانند به خواب بروند[۸].

دسته‌بندی منطقی

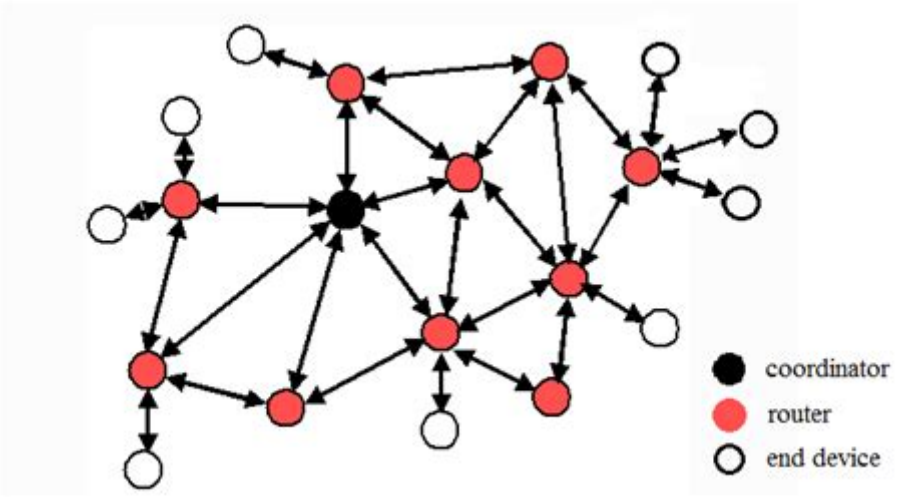
در این دسته‌بندی سه نوع دستگاه وجود دارد:

- هماهنگ‌کننده: ریشه درخت شبکه را تشکیل می دهد و می‌تواند به شبکه‌های دیگر متصل شود. در هر شبکه دقیقا یک هماهنگ‌کننده وجود دارد. هماهنگ‌کننده مسئول راه‌اندازی شبکه و انتخاب پارامترهای شبکه مانند کانال فرکانس رادیویی[۲۸]، شناسه یکتای شبکه و تنظیم سایر پارامترهای عملیاتی است. همچنین می‌تواند اطلاعات مربوط به شبکه و کلیدهای امنیتی را ذخیره کند[۸].
- مسیریاب: مسیریاب بعنوان گره میانی عمل می‌کند و داده‌ها را از دستگاه‌های دیگر منتقل می‌کند. مسیریاب می‌تواند به یک شبکه از قبل موجود متصل شود، همچنین می‌تواند درخواست اتصال سایر دستگاه‌ها را بپذیرد و نوعی فرستنده مجدد به شبکه باشد[۸].
- گره پایانی: این دستگاه می‌تواند دستگاه‌های کم‌مصرف یا با باتری باشد. آنها می‌توانند اطلاعات مختلفی را از حسگرها جمع‌آوری کنند و عملکرد کافی برای صحبت با والدین خود(هماهنگ‌کننده یا مسیریاب) را دارند اما نمی‌توانند داده‌ها را از دستگاه‌های دیگر ارسال کنند. این عملکرد

[24] Fully Functional Device(FFD)
[25] Routing
[26] Coordination
[27] Reduced Functional Device(RFD)
[28] Radio Frequency Channel

کاهش‌یافته امکان کاهش هزینه را فراهم می‌کند. این دستگاه‌ها مجبور نیستند تمام مدت بیدار بمانند، درحالیکه دستگاه‌های متعلق به دو دسته دیگر باید بیدار بمانند[۸].

در شکل ۲-۴ [۱۰] می‌توان یک شبکه زیگبی را که شامل یک هماهنگ‌کننده، چند مسیریاب و چند گره پایانی است را مشاهده کرد.



شکل ۲-۴: شبکه زیگبی شامل هماهنگ‌کننده، مسیریاب و گره‌پایانی [۱۰]

۲-۳-۴ توپولوژی

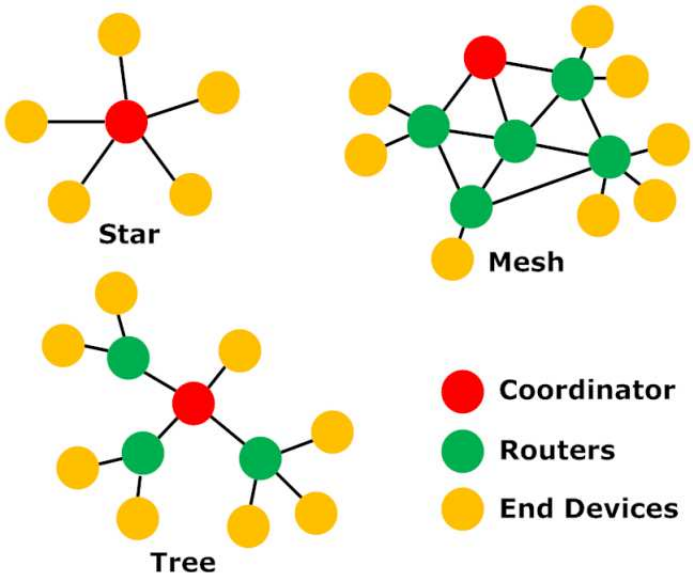
در زیگبی می‌توان سه توپولوژی زیر را داشت:

- ستاره: توپولوژی ستاره از یک هماهنگ کننده و تعدادی دستگاه پایانی تشکیل شده‌است. در توپولوژی ستاره، یک مدل شبکه master slave اتخاذ می شود که در آن master هماهنگ‌کننده و دستگاهی با عملکرد کامل است و slave دستگاهی با عملکرد کامل یا کاهش‌یافته است. دستگاه‌های انتهایی زیگبی بصورت فیزیکی و الکتریکی از یکدیگر جدا می‌شوند و اطلاعات را تنها از طریق هماهنگ‌کننده منتقل می‌کنند[۸].
- درخت خوشه‌ای[۲۹]: این توپولوژی مانند توپولوژی ستاره است، با این تفاوت که در این توپولوژی سایر دستگاه‌ها می‌توانند با یکدیگر ارتباط برقرار کنند تا دستگاه‌های بیشتری بتوانند به دستگاه‌های با عملکرد کامل و غیر هماهنگ‌کننده متصل شوند تا بتوان شبکه را از لحاظ جغرافیایی گسترش داد[۸].

[29] Cluster Tree

- مش[۳۰]: در توپولوژی مش، هر گره می‌تواند با هر گره دیگری در محدوده‌ی خود ارتباط برقرار کند. توپولوژی مش برای نگهداری پیچیده است اما نسبت به خطا مقاوم‌تر و قابل‌تحمل‌تر است[۸].

در شکل ۲-۵ [۱۱] می‌توان نقش گره‌های مختلف در توپولوژی‌های ذکرشده را مشاهده کرد.



شکل ۲-۵: توپولوژی‌های مختلف شبکه زیگبی [۱۱]

۲-۳-۵ دیجی بین‌الملل

دیجی بین‌الملل[۳۱] نام یکی از تولیدکنندگان ماژول‌های زیگبی است. ماژولی که ما در این پروژه استفاده می‌کنیم تولید این شرکت است. مدل این ماژول Digi XBee S2 است که می‌توان تصویر آن را در شکل ۲-۶ مشاهده کرد.

۲-۳-۶ طرز کار دستگاه‌های ایکس‌بی

دستگاه‌های ایکس‌بی از طریق هوا با یکدیگر ارتباط برقرار می‌کنند و پیام‌ها را به‌شکل بی‌سیم ارسال و دریافت می‌کنند. دستگاه‌ها فقط می‌توانند پیام‌ها را منتقل کنند ولی نمی‌توانند آنها را مدیریت کنند. با این‌حال، ماژول‌ها می‌توانند با دستگاه‌های هوشمند از طریق رابط سریال ارتباط برقرار کنند.

دستگاه‌های ایکس‌بی داده‌های ورودی سریال را از طریق هوا منتقل می‌کنند و هر چیزی را که بصورت بی‌سیم دریافت می‌شود به خروجی سریال می‌فرستند. چه برای مقاصد ارتباطی و چه صرفاً برای

[30] Mesh
[31] Digi International



شکل ۲-۶: ماژول زیگبی مدل XBee S2

پیکربندی دستگاه، ترکیبی از هر دو فرایند برای برقراری ارتباط لازم است. به این صورت، دستگاه‌های هوشمند مانند میکروکنترلر[۳۲]ها یا رایانه‌های شخصی می‌توانند آنچه را که دستگاه ارسال می‌کند کنترل کرده و پیام‌های دریافتی و ارسالی را مدیریت کنند[۱۲].

دو فرایند لازم برای ارتباط را می‌توان در شکل ۲-۷ [۱۲] دید که عبارتند از:

- ارتباط بی‌سیم: این ارتباط بین ماژول‌های ایکس‌بی انجام می‌شود. ماژول‌هایی که قرار است با هم کار کنند باید بخشی از یک شبکه باشند و باید از یک فرکانس رادیویی استفاده کنند. همه ماژول‌هایی که این الزامات را برآورده می‌کنند می‌توانند بصورت بی‌سیم با یکدیگر ارتباط برقرار کنند.

- ارتباط سریال: این ارتباط بین ماژول ایکس‌بی و دستگاه هوشمند متصل به آن از طریق رابط سریال صورت می‌گیرد.



شکل ۲-۷: فرایندهای لازم برای ارتباط دو ماژول ایکس‌بی [۱۲]

[32] Microcontroller

۲-۳-۷ ارتباط سریال ماژول‌های ایکس‌بی

دستگاه‌های ایکس‌بی می‌توانند از اتصال سریال محلی خود به روش‌های بسیار متفاوتی استفاده کنند. حالت عملیاتی نحوه ارتباط دستگاه میزبان با ماژول ایکس‌بی را از طریق رابط سریال تعیین می‌کند. این حالت‌ها به دو دسته تقسیم می‌شوند که در ادامه به شرح هر کدام می‌پردازیم.

حالت شفاف

هنگام کار در حالت شفاف[۳۳]، یک ماژول ایکس‌بی بعنوان جایگزین خط سریال عمل می‌کند. تمام داده‌های دریافت‌شده از طریق ورودی سریال بلافاصله از طریق هوا منتقل می‌شود. هنگامی که ماژول ایکس‌بی داده‌های بی‌سیم را دریافت می‌کند، از طریق رابط سریال به همان شکل محل دریافت‌شده ارسال می‌شود. در واقع، ارتباط در حالت شفاف مانند آن است که دو ماژول توسط یک سیم به هم متصل شده‌باشند[۱۲].

برای ارتباط دو ماژول ایکس‌بی، ماژول ارسال‌کننده به آدرس گیرنده نیاز دارد. هنگام کار در حالت شفاف، باید این آدرس را در ماژولی که در حال ارتباط است پیکربندی کرد. ماژول‌های ایکس‌بی می‌توانند آدرس ۶۴ بیتی کامل ماژول مقصد را ذخیره کنند[۱۲]. این آدرس باید در دو پارامتر نوشته شود: آدرس مقصد بالا[۳۴] و آدرس مقصد پایین[۳۵]. بعنوان مثال برای ارتباط دو ماژول در این حالت باید مثل شکل ۲-۸ [۱۲] دو ماژول را پیکربندی کرد.



شکل ۲-۸: ارتباط سریال در حالت شفاف [۱۲]

حالت شفاف محدودیت‌هایی دارد. بعنوان مثال، هنگام کار با چند ماژول، باید قبل از ارسال هر پیام، مقصد را پیکربندی کرد. با این حال، حالت شفاف به دلایل زیر راهی آسان برای شروع کار با دستگاه‌های ایکس‌بی فراهم می‌کند:

[33] Transparent Mode
[34] Destination Address High(DH)
[35] Destination Address Low(DL)

- عملیات بسیار ساده است.
- آنچه ارسال می‌شود دقیقا همان چیزی است که در ماژول دیگر دریافت می‌شود.
- سازگار با هر دستگاهی است که بتواند از طریق یک رابط سریال ارتباط برقرار کند.

حالت API

حالت API یک رابط ساختاریافته را فراهم می‌کند که در آن داده‌ها از طریق رابط سریال در بسته‌های سازماندهی‌شده و به ترتیب تعیین‌شده ارتباط برقرار می‌کنند. این روند اجازه می‌دهد بدون نیاز به تعریف پروتکل، ارتباط پیچیده بین دستگاه‌ها برقرار کرد[۱۲].

در حالت شفاف تمام داده‌های دریافت‌شده از طریق ورودی سریال برای ارسال رادیویی در صف قرار می‌گیرند و داده‌های دریافت‌شده به‌صورت بی‌سیم دقیقاً همان‌طور که دریافت شده‌اند، بدون اطلاعات اضافی به خروجی سریال ارسال می‌شوند[۱۲].

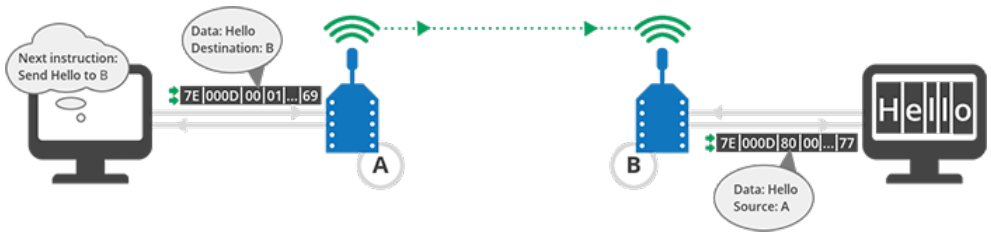
به دلیل این رفتار، دستگاه‌هایی که در حالت شفاف کار می‌کنند محدودیت‌هایی دارند:

- برای خواندن یا نوشتن پیکربندی یک دستگاه در حالت شفاف، ابتدا باید دستگاه را به حالت فرمان تغییر داد.
- اگر دستگاهی نیاز به انتقال پیام به دستگاه‌های مختلف داشته‌باشد، باید پیکربندی آن برای ایجاد یک مقصد جدید بروز شود. دستگاه باید برای تنظیم مقصد وارد حالت فرمان شود.
- دستگاهی که در حالت شفاف کار می‌کند نمی‌تواند منبع پیام بی‌سیمی را که دریافت کرده شناسایی کند. اگر نیاز به تمایز بین داده‌های دریافتی از دستگاه‌های مختلف باشد، دستگاه‌های فرستنده باید شامل اطلاعات اضافی شناخته‌شده توسط همه دستگاه‌ها باشند تا بتوان بعداً آنها را استخراج کرد.

حالت API راه بسیار آسان‌تری برای انجام اقدامات ذکرشده ارائه می‌دهد:

- از آنجایی که فریم‌های مختلفی برای اهداف مختلف (مانند پیکربندی و ارتباط) وجود دارد، می‌توان یک دستگاه را بدون وارد شدن به حالت فرمان پیکربندی کرد.
- از آنجایی که مقصد داده بعنوان بخشی از ساختار فریم API گنجانده شده‌است، می‌توان از حالت API برای انتقال پیام‌ها به چندین دستگاه استفاده کرد.

- فریم API همانطور که در شکل ۲-۹ [۱۲] نشان داده شده‌است، شامل منبع پیام است. بنابراین تشخیص اینکه داده‌ها از کجا آمده‌اند آسان است.



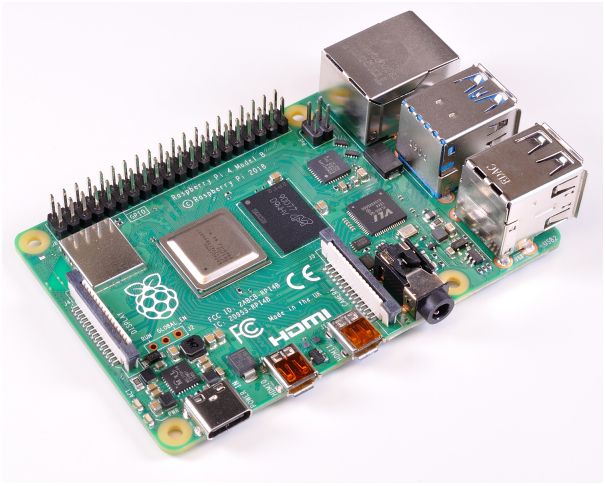
شکل ۲-۹: ارتباط سریال در حالت API [۱۲]

۲-۴ دروازه

برای دریافت، تجمیع، قراردادن داده‌ها در قالب مخصوص و ارسال به سرور به یک دستگاه بعنوان دروازه در هر کارخانه نیاز داریم. با توجه به اینکه ممکن است در یک لحظه چند گره انتهایی اطلاعات خود را برای دروازه ارسال کنند و با توجه به امکان نباز به انجام چند کار بصورت همزمان، نیاز به سخت‌افزاری با قابلیت چندرشته‌ای[۳۶] و منابع کافی برای جمع‌آوری و ارسال داده‌ها داریم. یکی از انتخاب‌های مناسب برای این کار رایانه‌های تک-بورد رزبری‌پای[۳۷] است.

مدل استفاده‌شده در این پروژه رزبری‌پای ۴ مدل بی[۳۸] است که تصویر آن در شکل ۲-۱۰ نشان داده شده است. رزبری‌پای ۴ مدل بی جدیدترین محصول از سری کامپیوترهای محبوب رزبری‌پای است. در مقایسه با نسل قبلی، سرعت پردازنده، عملکرد چندرسانه‌ای، حافظه و اتصال افزایش یافته است، در حالیکه سازگاری با محولات قبلی و مصرف انرژی مشابه حفظ شده است. برای کاربر، این مدل عملکرد قابل‌مقایسه‌ای با سیستم‌های رایانه شخصی x86 را ارائه می‌دهد. از ویژگی های کلیدی این محصول می‌توان به پردازنده چهار هسته‌ای ۶۴ بیتی با کارایی بالا، پشتیبانی از نمایشگر دوگانه با رزولوشن تا 4K از طریق یک جفت پورت micro-HDMI، رم تا ۴ گیگابایت،LAN بی‌سیم ۴/۲ و ۵ گیگاهرتز دو بانده، بلوتوث نسخه ۵ و درگاه USB 3.0 اشاره کرد[۱۳].

[36] Multi Threading
[37] Raspberry Pi Single-Board Computer
[38] Raspberry Pi 4 Model B



شکل ۲-۱۰: بورد زبری‌پای ۴ مدل بی

۲-۵ پایگاه داده‌ی سری زمانی

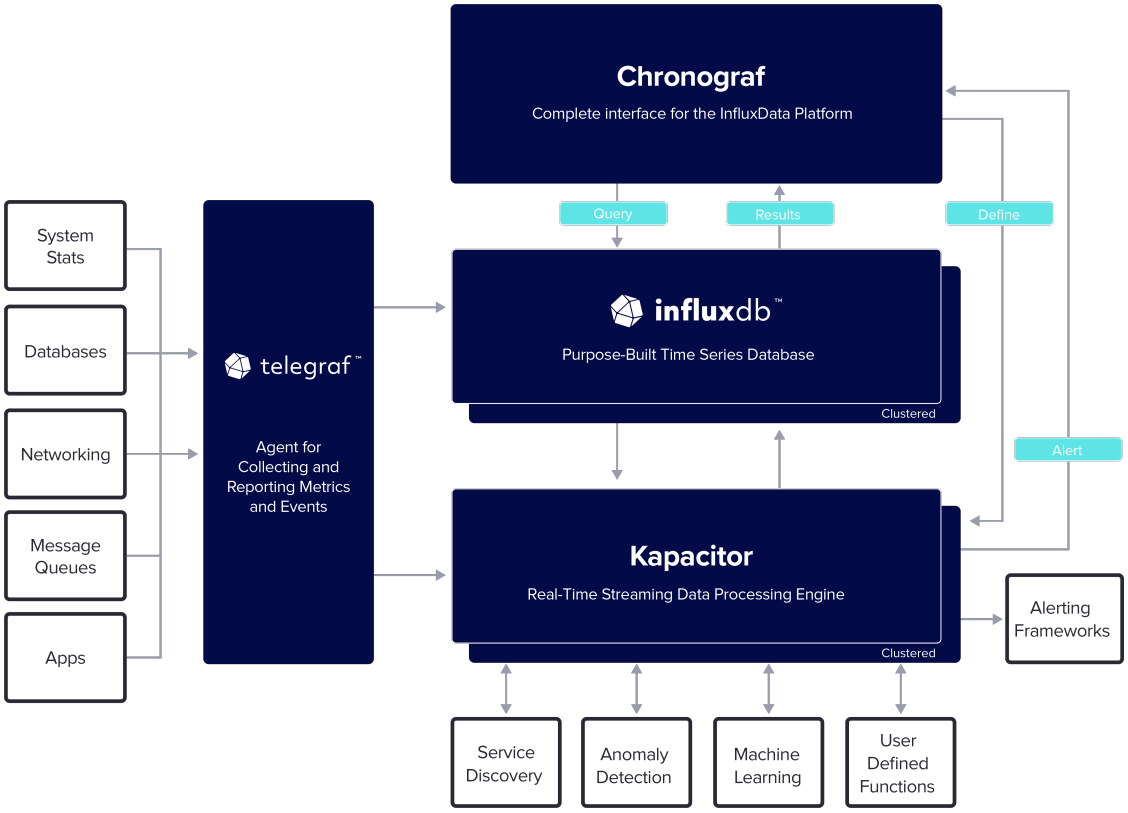
سری زمانی یک توالی مرتب‌شده از مقادیر یک متغیر در فواصل زمانی مساوی است. بنابراین سری زمانی یک توالی از داده‌های زمان گسسته است[۱۴]. داده‌های لرزش جمع‌آوری‌شده در این پروژه نیز از نوع سری زمانی هستند. جهت ذخیره‌سازی، پردازش و مدیریت بهینه این داده‌ها نیاز به یک پایگاه داده داریم.

InfluxDB یک پایگاه داده سری زمانی بدون طرحواره[۳۹] منبع باز با اجزای اختیاری منبع بسته است که توسط InfluxData توسعه یافته است. این به زبان برنامه‌نویسی گو[۴۰] نوشته شده‌است و برای مدیریت داده‌های سری زمانی بهینه شده‌است. این یک زبان پرس و جو[۴۱] مانند SQL را ارائه می دهد. نسخه منبع باز که در شکل ۲-۱۱ [۱۵] نشان داده شده‌است، یک پلتفرم پایگاه داده سری زمانی کامل را با خدمات مختلف از جمله هسته InfluxDB فراهم می‌کند و می‌تواند بر روی ابر و در محل روی یک گره واحد اجرا شود.

استفاده از پایگاه داده InfluxDB برتری‌هایی نسبت به سایر پایگاه داده‌ها دارد که باعث شد ما از آن برای ذخیره داده‌های لرزش استفاده کنیم. این برتری‌ها عبارتند از:

- طراحی اختصاصی برای داده‌های سری زمانی و عملکرد بهینه در خواندن و نوشتن این داده‌ها.
- افزودنی‌های رایگان و متعدد مانند HTTP API که باعث تسریع و راحتی توسعه می‌شود.

[39] Schemaless
[40] Go
[41] Query Language



شکل ۲-۱۱: پلتفرم متن باز پایگاه داده سری زمانی InfluxDB [۱۵]

- پشتیبانی از تعداد بالای زبان‌های برنامه‌نویسی.
- پشتیبانی از زبان پرس و جویی مانند SQL بنام InfluxQL که یادگیری و پیاده‌سازی را راحت‌تر می‌کند.

۲-۶ پیش‌پردازش

داده‌های جمع‌آوری‌شده در دروازه قبل از تحویل به مدل هوش مصنوعی نیاز به برخی پیش‌پردازش‌ها دارند. زبان برنامه‌نویسی انتخابی ما پایتون[۴۲] است. این زبان به دلیل راحتی در توسعه و وجود کتابخانه‌های زیاد در حوزه هوش مصنوعی انتخاب شده است.

کتابخانه‌ی استفاده‌شده برای پیش‌پردازش داده‌ها نامپای[۴۳] نام دارد. نامپای یک کتابخانه منبع باز[۴۴] توسعه‌یافته توسط جامعه[۴۵] است که یک شیء آرایه پایتون چندبعدی را به همراه توابع مرتبط با آرایه که

[42] Python
[43] NumPy
[44] Open Source
[45] Community-Developed

روی آن کار می‌کنند ارائه می‌دهد. نامپای در پردازش عددی از طریق ndarrayهای چندبعدی تخصص دارد، جایی که آرایه‌ها اجازه عملیات عنصربه‌عنصر[۴۶] یا پخش[۴۷] را می‌دهند. در صورت نیاز، جبر خطی را می‌توان بدون از قبل تغییردادن آرایه‌های نامپای استفاده کرد. علاوه بر این، اندازه آرایه‌ها را می‌توان بصورت پویا تغییر داد. این مسئله نگرانی‌هایی را که معمولاً برنامه‌نویسی سریع در زبان‌های دیگر را سردرگم می‌کند، از بین می‌برد[۱۶، ۱۷].

۲-۷ برنامه‌ی تحت وب

پس از دریافت داده‌های جمع‌آوری‌شده در دروازه و انجام مراحل پیش‌پردازش لازم، اکنون داده‌ها در اختیار مدل یادگیری ماشینی قرار می‌گیرد تا طول عمر مفید باقیمانده‌ی تجهیزات را پیش‌بینی کند. ما برنامه‌ی وبی را برای نمایش به متخصصان پیاده‌سازی کرده‌ایم تا بتوانند تعمیر و نگهداری بعدی خود را طبق پیش‌بینی برنامه‌ریزی کنند. برای پیاده‌سازی برنامه‌ی تحت وب ما از چارچوب ویو[۴۸] استفاده کرده‌ایم.

ویو یک چارچوب مترقی[۴۹] برای ساخت رابط‌های کاربری است. برخلاف سایر چارچوب‌های یکپارچه، ویو از ابتدا به گونه‌ای طراحی شده است که برای رویکرد افزایشی قابل‌پذیرش باشد. هسته‌ی کتابخانه فقط بر روی لایه‌ی نما[۵۰] متمرکز است و براحتی قابل‌انتخاب و ادغام با کتابخانه‌های دیگر یا پروژه‌های موجود است. از سوی دیگر، ویو قادر است برنامه‌های پیچیده تک صفحه‌ای را در ترکیب با ابزارهای نوین و کتابخانه‌های پشتیبان ایجاد کند[۱۸].

ویو دو ویژگی اصلی دارد:

- تفسیر اعلامی[۵۱]: ویو HTML استاندارد را با استفاده از نحو template گسترش می‌دهد که امکان توصیف صریح خروجی HTML بر اساس وضعیت جاوا اسکریپت[۵۲] را فراهم می‌کند.
- واکنش‌پذیری[۵۳]: ویو بطور خودکار وضعیت جاوا اسکریپت را دنبال می‌کند و در صورت تغییر،

[46] Element by Element
[47] Broadcast
[48] Vue Framework
[49] Progressive
[50] View Layer
[51] Declarative Rendering
[52] JavaScript
[53] Reactivity

مدل سند شیء[۵۴] را بروزرسانی می‌کند.

ویو چارچوب و اکوسیستمی است که اکثر ویژگی‌های مشترک مورد نیاز در توسعه سمت کاربر را پوشش می‌دهد. اما وب بسیار متنوع است. چیزهایی که در وب می‌سازیم ممکن است از نظر شکل و مقیاس بسیار متفاوت باشند. با توجه به این موضوع، ویو به‌شکلی طراحی شده است که منعطف باشد. اما با وجود انعطاف‌پذیری دانش اصلی در مورد نحوه عملکرد ویو در همه کاربردها یکسان است[۱۸]. این ویژگی‌ها چارچوب ویو را به چارچوبی کاربردی، کم‌حجم و آسان برای یادگیری تبدیل می‌کند.

۲-۸ جمع‌بندی و نتیجه‌گیری

در این بخش، به تکنولوژی‌ها، چارچوب‌ها و قطعات استفاده‌شده در پروژه پرداختیم. هریک از آنها را معرفی کرده و سپس ویژگی‌ها و دلیل انتخابشان را بیان کردیم. ابتدا درباره حسگر استفاده‌شده برای اندازه‌گیری لرزش صحبت کردیم. سپس به سراغ گره‌های انتهایی و بورد آردوینوی انتخاب‌شده رفتیم. بعد از آن پروتکل ارتباطی زیگبی، ویژگی‌ها، طرز کار و ماژول ایکس‌بی را معرفی کردیم. سپس دروازه و بورد رزبری‌پای استفاده‌شده را شناختیم. پایگاه داده‌ی InfluxDB که مناسب داده‌های سری زمانی بود را معرفی کردیم. برای پیش‌پردازش داده‌ها از کتابخانه نامپای گفتیم و در انتها نیز چارچوب ویو که برای پیاده‌سازی برنامه تحت وب استفاده شده بود را مرور کردیم.

[54] Document Object Model

فصل سوم

پیاده‌سازی سیستم

در این قسمت نحوه‌ی اجرای پروژه را بطور کامل توضیح خواهیم داد. در هر قسمت ساختار فایل‌ها، نحوه‌ی پیاده‌سازی و چالش‌هایی که با آن مواجه بودیم به‌همراه نحوه‌ی حل آنها را توضیح خواهیم داد.

۳-۱ معماری کلی سیستم

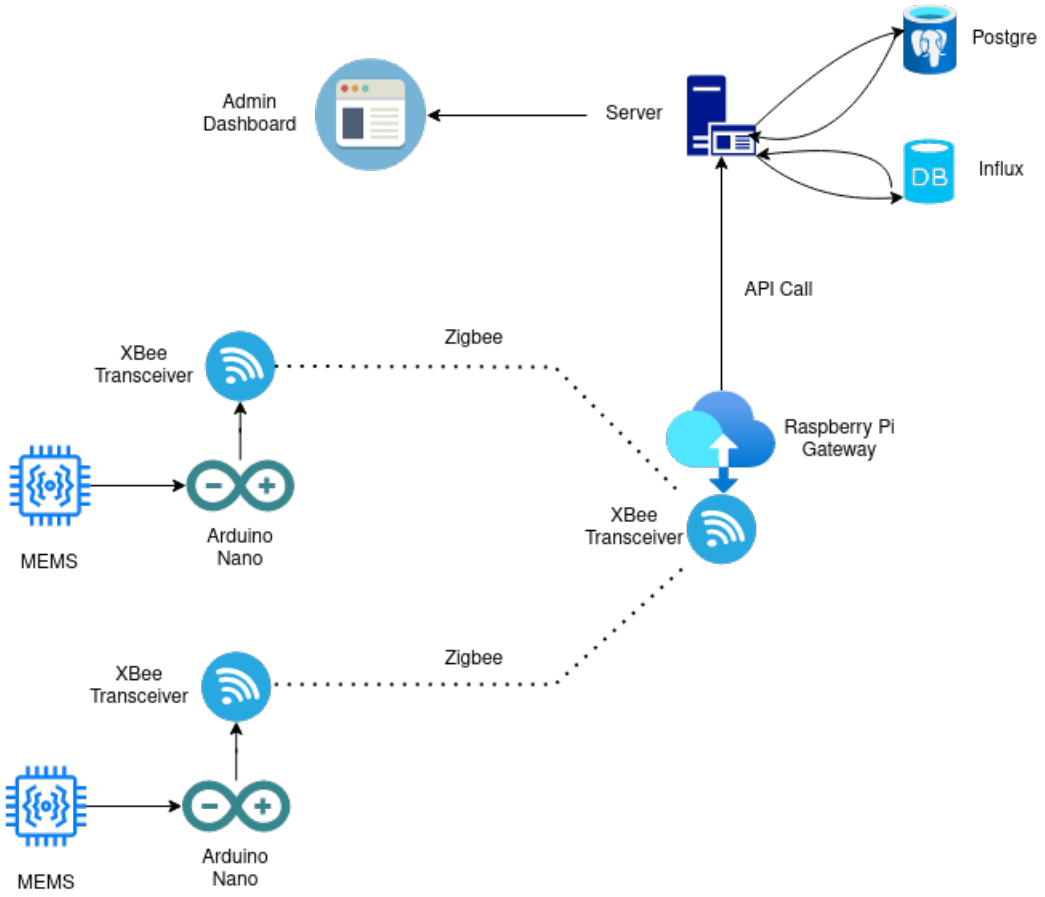
برای درک بهتر این پروژه، نمایی کلی از سیستم و نحوه عملکرد آن را همانطور که در شکل ۳-۱ نشان داده شده است، ارائه خواهیم داد. این پروژه در چندین محل صنعتی مستقر خواهد شد. در هر کارخانه چندین میکروکنترلر آردوینو، با یک سنسور لرزش و یک فرستنده گیرنده زیگبی متصل، روی هر قطعه از تجهیزات نصب شده است. داده‌ها در زمان‌های برنامه‌ریزی‌شده حس می‌شوند و سپس به دروازه‌ای در کارخانه ارسال می‌شوند. دروازه اطلاعاتی را که از فرستنده‌گیرنده‌های زیگبی می‌آید را جمع‌آوری می‌کند تا زمانی که به یک آستانه‌ی از پیش تعریف‌شده برسد. سپس دروازه با سرور مرکزی احراز هویت می‌شود و سپس داده‌های جمع‌آوری‌شده را با پروتکل HTTP به سرور ارسال می‌کند. در سرور مراحل پیش‌پردازش لازم انجام می‌شود و سپس داده‌ها در پایگاه داده‌ی سری زمانی ذخیره می‌شوند. سپس مدل یادگیری ماشین با داده‌های قبلی و جدید اجرا می‌شود تا پیش‌بینی دقیق‌تری درباره طول عمر مفید باقیمانده تجهیزات به ما ارائه دهد. در صورت درخواست، تمام داده‌های تحلیلی و پیشبینی‌ها از طریق داشبورد مدیریت قابل‌دسترسی است. متخصصان می‌توانند از این داده‌ها برای برنامه‌ریزی زمان نگهداری مناسب استفاده کنند.

۳-۲ گره انتهایی و آردوینو

برای پیاده‌سازی گره انتهایی، سه بخش را باید پیاده‌سازی میکردیم که عبارتند از:

- خواندن از حسگر و نمونه‌برداری
- تشکیل فریم زیگبی و نوشتن در درگاه سریال
- خوابیدن و بیدارشدن در بازه‌های زمانی مشخص

در ادامه هر بخش را توضیح می‌دهیم.



شکل ۳-۱: نمای کلی سیستم

۳-۲-۱ خواندن از حسگر و نمونه‌برداری

حسگر ADXL345 چندین پارامتر دارد که باید در زمان راه‌اندازی تنظیم شوند. برای انجام پیکربندی و همچنین خواندن داده‌های حسگرها بدون نیاز به استفاده مستقیم از پروتکل I2C، از کتابخانه[1] استفاده کرده‌ایم. یکی از پارامترها محدوده شتاب است که با تابع setRange تنظیم می‌شود. محدوده مجاز ۲، ۴، ۸ یا ۱۶ برابر شتاب گرانش زمین است. اما هرچه دامنه بزرگتر شود، داده‌ها دقت کمتری پیدا می‌کنند. از آنجایی که یک قطعه از تجهیزات حتی زمانی که قدیمی باشد با شتاب بالا نمی‌لرزد و برای پیش‌بینی‌های دقیق‌تر به دقت نیاز است، تصمیم گرفتیم که محدوده را روی دو برابر شتاب گرانش زمین تنظیم کنیم.

پارامتر دیگر نرخ داده است که باید در زمان راه‌اندازی با استفاده از تابع setDataRate تنظیم شود. نرخ داده برای تنظیم نرخ ارتباط بین حسگر و بورد آردوینو از طریق پروتکل I2C استفاده می‌شود. بنابراین مهم است که نرخ داده با رسانه‌ی ارتباطی سازگار باشد. از آنجایی که اگر نرخ داده بالاتر از نرخی باشد که رسانه قادر به انجام آن است، برخی از داده‌ها از بین می‌روند. بدلیل برخی چالش‌های دیگر

[1] Adafruit_ADXL345_U.h

که در ادامه در مورد آنها صحبت خواهد شد، نرخ را روی ۱۰۰ هرتز تنظیم کرده‌ایم. برای نمونه‌برداری از داده‌ها در فرکانس نمونه‌برداری مورد نظر، تأخیر نمونه‌برداری برای تأخیر بین هر نمونه حسگر محاسبه می‌شود.

برای هر نمونه، از یک رویداد برای بدست‌آوردن شتاب در سه محور x، y، z استفاده می‌کنیم. سپس داده‌ها در یک آرایه ذخیره می‌شوند تا بعداً قالب‌بندی و با پروتکل زیگبی ارسال شوند. اگر فرمت و ارسال در حین نمونه‌برداری انجام شود، بدلیل تاخیر در فرمت، تشکیل و ارسال فریم‌ها از طریق درگاه سریال، فرکانس نمونه‌برداری صحیح نخواهد بود.

ما چالش دیگری در این شرایط بوجود می‌آید. با توجه به محدودیت حافظه موجود در دستگاه آردوینو نانو، فضای ایجاد آرایه‌ها و ذخیره داده‌ها نیز محدود است. به همین دلیل است که نمی‌توانیم به فرکانس‌های نمونه‌برداری بالا دست پیدا کنیم.

۳-۲-۲ تشکیل فریم زیگبی و نوشتن در درگاه سریال

همانطور که در بخش ۲-۳-۷ گفته شد، در حالت شفاف پیام‌ها به‌شکل مستقیم منتقل می‌شوند اما در حالت API نیاز به تشکیل فریم دارند. ابتدا سراغ استفاده از کتابخانه‌های آماده برای تشکیل و نوشتن فریم بر روی درگاه سریال رفتیم. یکی از آنها کتابخانه xbee-arduino[۲] بود. اما این کتابخانه مشکلات فراوانی داشت که سبب عدم ارسال فریم توسط ماژول می‌شد. یکی از این مشکلات محاسبه اشتباه چک‌سام و عدم مطابقت با استاندارد بود. مشکل دیگر نیز در نوشتن بر روی درگاه سریال بود. همه این مشکلات باعث شد تا ما کتابخانه خود را پیاده‌سازی کنیم. اگرچه این پیاده‌سازی کلی نیست و بعضا برای این پروژه انجام شده است، اما کارکرد مورد انتظار را دارد.

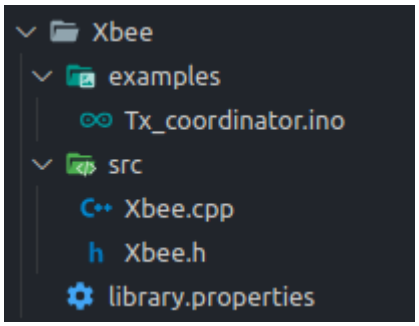
کتابخانه Xbee

ساختار فایل کتابخانه ایکس‌بی در شکل ۳-۲ نشان داده شده است. ساختار طبق استانداردهای آردوینو[۳] برای ایجاد یک کتابخانه است. فایل‌های اصلی در پوشه src قرار دارند، یک فایل هدر که توابع، ورودی‌ها و خروجی‌ها را تعریف می‌کند و یک فایل سی‌پلاس‌پلاس برای پیاده‌سازی و تکمیل آن توابع. در پوشه نمونه‌ها، فایلی با فرمت ino وجود دارد که کاربرد اصلی کتابخانه برای انتقال یک فریم در حالت API را نشان می‌دهد. یک فایل library.properties در ریشه پوشه وجود دارد. این فایل برای تشریح برخی از

[2] https://github.com/andrewrapp/xbee-arduino

[3] https://arduino.github.io/arduino-cli/0.33/library-specification/

ویژگی‌های این کتابخانه از جمله نام، نویسنده، توضیحات، نسخه کتابخانه، معماری‌های سازگار و اینکه این کتابخانه به کدام کتابخانه‌ها وابستگی دارد استفاده می‌شود. این فایل به IDE آردوینو کمک می‌کند تا کتابخانه و وابستگی‌های آن را نصب کند.



شکل ۳-۲: ساختار فایل کتابخانه ایکس‌بی

ساختار فریم API

ساختار کلی فریم API در شکل ۳-۳ [۱۲] نشان داده شده است. فریم با یک بایت جداکننده شروع می‌شود، سپس با ۲ بایت برای نشان‌دادن طول فریم ادامه می‌یابد و با یک بایت چک‌سام پایان می‌یابد. بایت‌های بین طول و فیلد چک‌سام را داده‌های فریم می‌نامند که برای هر نوع بسته API متفاوت است. بایت اول فیلد طول مهم‌ترین بایت[۴] و بایت بعدی کم‌اهمیت‌ترین[۵] است. اولین بایت از داده‌های فریم نشان‌دهنده نوع فریم است. چک‌سام برای کمک به تست یکپارچگی داده‌ها استفاده می‌شود. اگر مقدار چک‌سام به اشتباه محاسبه شده‌باشد یا با چک‌سام واقعی داده‌های درون فریم متفاوت باشد، فریم کنار گذاشته و نادیده گرفته می‌شود.

Start delimiter	Length		Frame data							
			Frame type	Data						Checksum
1	2	3	4	5	6	7	8	9	...	n	n+1
0x7E	MSB	LSB	API frame type	Frame-type-specific data							Single byte

شکل ۳-۳: ساختار کلی فریم API [۱۲]

روش صحیح محاسبه چک‌سام فریم به شرح زیر است:

۱. تمام بایت‌های بسته به استثنای محدودکننده شروع و طول با هم جمع می‌شوند.

[4] Most Significant Byte(MSB)

[5] Least Significant Byte(LSB)

۲. از نتیجه فقط پایین‌ترین ۸ بیت نگه داشته می‌شود.

۳. مقدار بدست‌آمده از 0xFF کم می‌شود.

فریم درخواست ارسال

فریم‌های API بسته به عملکردشان از چندین نوع فریم پشتیبانی می‌کنند. در این بخش فقط در مورد فریم درخواست ارسال که برای ما مهم است صحبت خواهیم کرد.

بخش‌های مختلف داده‌های فریم درخواست ارسال در جدول ۳-۱ به همراه کاربرد آمده است.

جدول ۳-۱: بخش‌های مختلف داده‌های فریم درخواست ارسال [۱۲]

نام بخش	شماره بایت	نمونه	توضیحات
نوع فریم	۳	0x10	بایت 0x10 نشان‌دهنده فریم درخواست ارسال است.
شماره فریم	۴	0x01	فریم داده را به گیرنده برای ارسال فریم وضعیت ارسال می‌شناساند. تنظیم‌کردن با مقدار ۰ پاسخ را غیرفعال می‌کند.
آدرس ۶۴ بیتی مقصد	۵ تا ۱۲	0x0000000000000000	آدرس ۶۴ بیتی مقصد را مشخص می‌کند. آدرس مقاصد خاص: هماهنگ‌کننده: 0x0000000000000000 همه‌پخشی[۶]: 0x000000000000FFFF نامعلوم(اگر آدرس مقصد را ندانیم): 0xFFFFFFFFFFFFFFFF
آدرس ۱۶ بیتی مقصد	۵ تا ۱۲	0x0000	آدرس ۱۶ بیتی مقصد را مشخص می‌کند. آدرس مقاصد خاص: هماهنگ‌کننده: 0x0000 نامعلوم یا همه‌پخشی: 0xFFFE
شعاع پخش	۱۵	0x00	حداکثر تعداد پرش‌هایی که انتقال همه‌پخشی می‌تواند پخش شود را تنظیم می‌کند. اگر روی ۰ تنظیم شود، شعاع پخش روی حداکثر مقدار پرش تنظیم می‌شود.
انتخاب‌ها	۱۶	0x00	گزینه‌های انتقال پشتیبانی‌شده: 0x01 - غیرفعال‌کردن تلاش ارسال مجدد فعال‌کردن رمزگذاری APS(با این کار حداکثر بایت‌های داده ارسالی تا ۴ بایت کاهش می‌یابد) - 0x20 0x40 - استفاده از مهلت زمان طولانی برای ارسال
داده ارسالی	۱۷ به بعد	0x...	تا ۲۵۵ بایت داده ارسالی برای مقصد

داده ارسالی

برای استفاده بهینه از فضای پیام در هر فریم، تصمیم گرفتیم ارتعاشات حسگر را به صورت زیر ارسال کنیم:

- هر داده با دقت ۳ رقم اعشار ارسال می‌شود.
- برای جداسازی ارتعاش بر اساس محور در پیام ارسالی، از حروف x، y و z استفاده نمی‌کنیم.
- اولین بایت همیشه شناسه اندازه‌گیری را نشان می‌دهد. پس از آن پنج بایت اول نشان‌دهنده محور ایکس، پنج بایت بعدی نشان‌دهنده محور وای و پنج بایت بعدی نشان‌دهنده محور زد است. هر داده در پنج بایت نمایش داده می‌شود، برای مثال عدد ۳/۴۵۲- بصورت {'-','3','4','5','2'} نمایش داده می‌شود.

در سمت گیرنده، داده‌ها باید به حالت اولیه تبدیل شوند.

فایل Xbee.cpp و توابع مهم

برای تنظیم آدرس مقصد برای هر فریم کلاس XbeeDestAddress را تعریف کرده‌ایم. این کلاس شامل فیلد ۶۴ بیتی و ۱۶ بیتی برای آدرس است. با توجه به اینکه به آرایه‌ای از بایت‌ها برای تشکیل فریم نیاز داریم، توابعی متناظر برای گرفتن هر یک از فیلدها نیز نوشته شده است.

برای تشکیل فریم درخواست انتقال کلاس XbeeRequest تعریف شده است. تابع constructFrame برای ایجاد فریم با استفاده از شناسه فریم و داده مورد نظر نوشته شده است. این تابع فریم را طبق بخش ۳-۲-۲ ایجاد کزده و فریم تشکیل‌شده برگردانده می‌شود. با استفاده از تابع writeFrameToSerial نیز می‌توان فریم تشکیل‌شده را بر روی درگاه سریال دلخواه نوشت.

۳-۲-۳ خوابیدن و بیدارشدن در بازه‌های زمانی مشخص

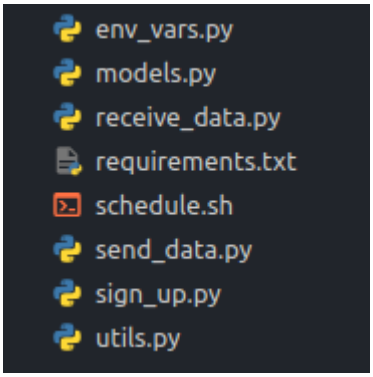
برای کاهش مصرف انرژی، حسگر تنها در زمان‌های مشخص لرزش را احساس می‌کند. در سایر زمان‌ها بورد آردوینو برای کاهش مصرف انرژی به خواب می‌رود. برای این کار از کتابخانه LowPower.h استفاده می‌کنیم. با استفاده از تابع powerDown آردوینو به خواب می‌رود. برای بیدارشدن در زمان مشخص، آردوینو از زمان‌بند داخلی خود[۷] استفاده می‌کند. اما این تایمر بعد از ۸ ثانیه سرریز شده و بورد بیدار

[7] Watchdog Timer

می‌شود. برای زمان‌های بیشتر باید به محض بیدارشدن مجدد تا رسیدن به زمان مورد نظر، آن را مجددا به خواب ببریم. این کار در تابع longSleep انجام شده است.

۳-۳ دروازه

در دروازه ما از زبان برنامه‌نویسی پایتون برای دریافت فریم‌های زیگبی، ذخیره‌کردن و ارسال داده‌ها به سرور مرکزی پس از رسیدن به آستانه استفاده می‌کنیم. ساختار پروژه دروازه در شکل ۳-۴ نشان داده شده است. در این بخش به بررسی هر یک از فایل ها می‌پردازیم.



شکل ۳-۴: ساختار پروژه دروازه

۳-۳-۱ فایل env_vars.py

این فایل از بسته load_dotenv برای خواندن متغیرهای محیطی استفاده می‌کند. برخی از این متغیرها برای برقراری ارتباط با سرور استفاده می‌شوند مانند شماره پورت سرور و آیپی سرور. سایر آنها برای برقراری ارتباط با دستگاه ایکس‌بی استفاده می‌شوند. این متغیرهای محیطی به عنوان متغیرهای پایتون تنظیم می‌شوند و سپس در ماژول‌های دیگر پایتون استفاده می‌شوند.

۳-۳-۲ فایل models.py

در این فایل ما سه کلاس برای نمایش داده‌های ارتعاش، اندازه گیری‌ها و هر گره انتهایی تعریف کرده‌ایم.

کلاس داده ارتعاش شامل داده‌های لرزش در سه محور ثبت‌شده توسط حسگر می‌باشد. هر اندازه‌گیری بسته به فرکانس نمونه‌برداری شامل چندین داده ارتعاش است. همچنین شامل یک شناسه و زمانی است که اندازه‌گیری در آن انجام شده است. از آنجایی که برای برقراری ارتباط داده‌های ارتعاش با زیگبی

قطعه‌بندی[8] لازم است و زمانی بین ارسال هر فریم لازم است، تابع thread_handler تعریف شده است تا در صورت اتمام زمان اتصال یا از بین رفتن برخی فریم‌ها، اندازه‌گیری از داده‌های دریافتی قبلی بطور خودکار تکمیل شود. پس از آن داده‌های جدید ذخیره می‌شوند.

هر گره دارای یک شناسه است که با شناسه فرستنده ارائه‌شده در هر فریم زیگبی مشخص می‌شود. همچنین هر گره شامل چندین اندازه‌گیری است. هنگامی که اندازه‌گیری جدیدی به لیست اندازه‌گیری‌ها اضافه می‌شود، زمان‌بند رشته‌ای[9] با دوره وقفه مشخص و تابع thread_handler شروع می‌شود.

۳-۳-۳ فایل receiver.py

در این فایل سروری تعریف می‌کنیم که بطور بی‌نهایت به درگاه USB متصل به دستگاه ایکس‌بی گوش می‌دهد. برای خواندن داده‌ها و استخراج داده هر فریم از کتابخانه digi-xbee استفاده می‌کنیم. برای مقداردهی اولیه یک نمونه ایکس‌بی، درگاه و نرخ باود[10] ارتباط لازم است. در تابع decode_sensor_data، شناسه اندازه‌گیری و داده‌های ارتعاش را استخراج می‌کنیم. در تابع receive، به منتظر دریافت پیام جدید می‌مانیم. داده‌های پیام را به گره و اندازه‌گیری مربوطه اضافه می‌کنیم و سپس داده‌های ذخیره‌شده را بروز می‌کنیم.

۴-۳-۳ فایل requirements.txt

این فایل برای تعیین بسته‌ها، نسخه‌ها و وابستگی‌های لازم استفاده می‌شود. سپس از این فایل برای نصب بسته‌های مشخص‌شده با مدیریت بسته پایتون پیپ[11] استفاده می‌شود.

۵-۳-۳ فایل schedule.sh

آستانه تعریف‌شده بر اساس زمان است. این اسکریپت در کنار گیرنده اجرا می‌شود. در این اسکریپت یک کار crontab برنامه‌ریزی می‌شود که هر ۱۵ دقیقه اجرا شود. این زمان‌بندی با استفاده از علامت * * 15/* * * انجام می‌شود که به معنای اجرای اسکریپت ارسال داده پایتون در هر دقیقه قابل‌تقسیم بر ۱۵ است.

[8] Fragmentation
[9] Thread Timer
[10] Buad Rate
[11] pip

۳-۳-۶ فایل send_data.py

در این فایل اطلاعات ذخیره‌شده در حافظه بارگذاری می‌شود. برای سرور و فراخوانی نقاط انتهایی API از کتابخانه requests استفاده می‌کنیم. برای دریافت رمز احراز هویت JWT برای ارسال داده‌های ارتعاشی، یک درخواست پست با آدرس مک دستگاه و رمز عبور به سرور ارسال می‌شود تا رمز احراز هویت را دریافت کند. پس از آن داده‌های لرزش فرمت‌شده بصورت جیسان[۱۲] در بدنه درخواست به همراه هدر احراز هویت به سرور ارسال و فایل ذخیره نیز پاک می‌شود.

۳-۳-۷ فایل signup.py

این فایل برای ثبت دروازه در سرور با رمز عبور و آدرس مک مربوطه استفاده می‌شود.

۳-۳-۸ فایل utils.py

این فایل شامل برخی از توابع کاربردی مانند دریافت آدرس مک دستگاه، دریافت زمان جاری در شکل مورد نظر، بروز رسانی داده‌های ذخیره‌شده و تعریف کلاس مدل جیسان برای کدکردن داده‌ها می‌باشد.

۳-۴ پیش‌پردازش

داده‌های جمع‌آوری‌شده از حسگر در هر سه محور می‌توانند تحت تاثیر شتاب گرانش زمین باشند. همچنین شتاب اندازه‌گیری‌شده توسط حسگرهای کم‌هزینه MEMS نیز اکثرا تحت تاثیر یک مقدار غیرصفر دچار انحرافاتی می‌شوند که منجر به افزایش یا کاهش اندازه‌گیری‌ها می‌شود[۵]. برای حذف انحرافات و شتاب گرانش ناخواسته، داده‌ها را در مرحله پیش‌پردازش هنجارسازی می‌کنیم. برای حذف ناهنجاری‌ها مطابق برابری (۳-۱) میانگین هر محور را ار هر داده کم می‌کنیم. لازم به ذکر است که $\hat{a}^l_{nm}$ نماد ماتریس هنجارشده برای شتاب در سه محور، گره n و اندازه‌گیری mام است.

$$\hat{a}^l_{nm} = a^l_{nm} - \sum_{k=1}^{K} \frac{a^l_{nmk}}{K} \qquad (۳-۱)$$

[12] JavaScript Object Notation(JSON)

۳-۵ پایگاه داده سری زمانی

مدل داده InfluxDB داده‌های سری زمانی را در سطل‌ها[۱۳] و اندازه‌گیری‌ها سازماندهی می‌کند. یک سطل می‌تواند چندین اندازه‌گیری داشته باشد. اندازه‌گیری گروه‌بندی منطقی برای داده‌های سری زمانی است که شامل چندین برچسب[۱۴] و فیلد است. همه نقاط در یک اندازه‌گیری معین باید برچسب‌های یکسانی داشته باشند. برچسب‌ها جفت‌های کلید-مقدار با مقادیری متفاوت هستند که اغلب تغییر نمی‌کنند و برای ذخیره ابرداده[۱۵] برای هر نقطه در نظر گرفته شده‌اند. فیلدها جفت‌های کلید-مقدار هستند که مقادیر آنها در طول زمان تغییر می‌کنند. زمان مرتبط با داده‌ها نیز ذخیره می‌شوند و هنگام پرس‌وجو داده‌ها برحسب آن مرتب می‌شوند[۱۹].

برای ارتباط با پایگاه داده از کلاینت پایتون influxDB استفاده می‌کنیم. برای شروع یک اتصال به نام سطل، سازمان، نشانه و نشانی نیاز داریم. برای نوشتن داده‌ها از تابع write_api و برای پرس‌وجوی داده‌ها در پایگاه داده از query_api استفاده می‌کنیم.

برای نوشتن داده‌ها باید نقاطی را که می‌خواهیم اضافه کنیم، مشخص کنیم. نقطه یک داده واحد است که با اندازه‌گیری، کلیدهای برچسب، مقادیر برچسب، کلید فیلد و زمان مشخص می‌شود. برای پرس‌وجوی داده‌ها، از عملگر لوله جلو[۱۶] برای اتصال خروجی یک تابع بعنوان ورودی تابع بعدی استفاده می‌کند[۱۹]. سه تابع اصلی برای پرس‌وجوکردن داده‌ها وجود دارد:

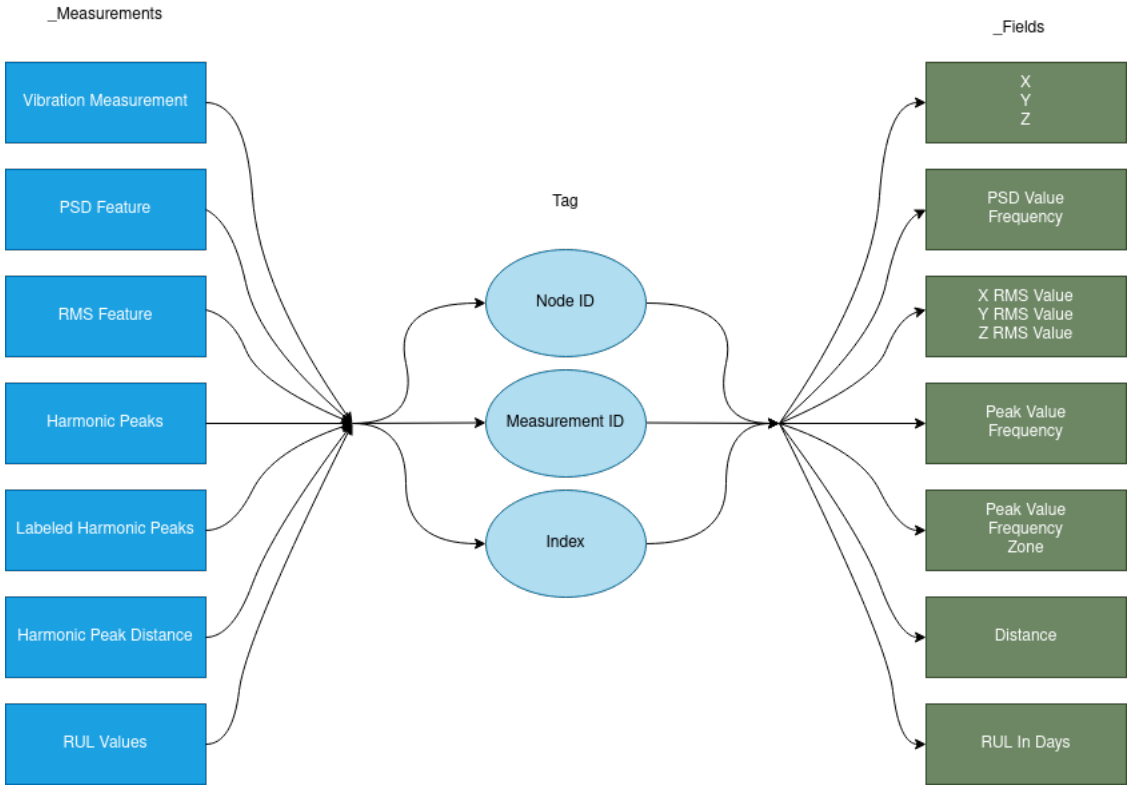
- from(): داده‌ها را از یک سطل جستجو می‌کند.
- range(): داده‌ها را بر اساس محدوده‌ی زمانی فیلتر می‌کند. برای جستجو به پرس‌وجوهای محدود به یک بازه‌ی زمانی خاص نیاز داریم.
- filter(): داده‌ها را بر اساس مقادیر ستون فیلتر می‌کند. هر سطر با r و هر ستون با ویژگی r نشان داده می‌شود. می‌توان چند فیلتر متوالی اعمال کرد.

تابع دیگری که استفاده کرده‌ایم pivot() است که برای تبدیل داده‌ها به یک طرح رابطه‌ای[۱۷] بر اساس زمان استفاده می‌شود.

[13] Buckets
[14] Tag
[15] Meta data
[16] Pipe-Forward(|>)
[17] Relational Schema

۳-۵-۱ ساختار کلی

ساختار کلی پایگاه داده طراحی‌شده در این پروژه در شکل ۳-۵ نشان داده شده است. هر یک از ویژگی‌های استخراج‌شده در سمت چپ در اندازه‌گیری‌های مختلف قرار گرفته‌اند. در وسط تصویر نیز برچسب‌ها که بین همه اندازه‌گیری‌ها مشترک است قرار گرفته است. در سمت راست تصویر نیز فیلدهای هر اندازه‌گیری آورده شده است. در ادامه به توضیح توابع نوشتن و پرس‌وجوی اندازه‌گیری‌ها می‌پردازیم.



شکل ۳-۵: ساختار کلی پایگاه داده سری زمانی

۳-۵-۲ اندازه‌گیری‌ها و توابع مربوط به نوشتن و گرفتن داده

در هر اندازه‌گیری، فیلدهای مربوط را ذخیره می‌کنیم. فیلدهای هر داده ثبت می‌شود و با یک گره و یک اندازه‌گیری همراه است. همچنین یک شماره به هر نمونه در اندازه‌گیری بعنوان شناسه داده می‌شود.

برای نوشتن داده‌ها در پایگاه داده توابع write_${measurement name} پیاده‌سازی شده است که یک نقطه جدید برای هر داده با شناسه اندازه‌گیری، شناسه گره و شماره بعنوان برچسب و فیلدهای مربوطه ایجاد می‌کند.

برای پرس‌وجوی داده، توابع get_${measurement name} پیاده‌سازی شده است. پرس‌وجو بگونه‌ای

نوشته شده است که اگر شناسه اندازه‌گیری یا شناسه گره ارائه شود، فقط داده‌های مرتبط برگردانده می‌شود. در هر پرس‌وجو، ما تمام داده‌ها را از اندازه‌گیری مربوط انتخاب می‌کنیم و در صورت وجود شناسه گره یا اندازه‌گیری فیلتر می‌کنیم. سپس زمان فیلدها، مقادیر آنها و برچسب‌ها را نگه می‌داریم و از تابع pivot برای تبدیل نتایج به طرح رابطه‌ای استفاده می‌کنیم. بعنوان نمونه، کد پرس‌وجوی vibration_measurement در شکل ۳-۶ نشان داده شده است.

```
filter_by_node: str = f'|> filter(fn:(r) => r.nodeId == "{nodeId}")'
filter_by_measurement: str = f'|> filter(fn:(r) => r.measurementId == "{measurementId}")'

query: str = f'from(bucket:"{INFLUXDB_BUCKET}")\
|> range(start: 0)\
|> filter(fn:(r) => r._measurement == "vibration_measurement")\
{filter_by_node if nodeId is not None else ""}\
{filter_by_measurement if measurementId is not None else ""}\
|> keep(columns: ["_time", "_field", "_value", "nodeId", "measurementId", "index"])\
|> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")\
|> yield()'
```

شکل ۳-۶: کد پرس‌وجوی vibration_measurement

۳-۵-۳ توابع گرفتن شناسه گره و اندازه‌گیری

در این توابع برای گرفتن شناسه‌ها به تنهایی، برچسب مورد نظر را نگه می‌داریم و جهت جلوگیری از تکرار با استفاده از تابع unique() مقادیر یکتای شناسه را برمی‌گردانیم.

۳-۵-۴ تابع get_starting_service_date

این تابع برای دریافت زمان اولین داده ثبت‌شده بعنوان مبدا زمانی برای محاسبه زمان سرویس‌شدن تجهیزات نوشته شده است.

۳-۵-۵ تابع حذف داده‌ها و حافظه پنهان

برای حذف از کلاس DeleteApi کتابخانه استفاده می‌کنیم. جهت حذف حافظه پنهان، همه اندازه‌گیری‌ها را، بجز آنهایی که خودمان تعریف کرده‌ایم، تا زمان فعلی حذف می‌کنیم. همچنین برای حذف همه داده‌ها با استفاده از تابع clear_vibration_data، همه اندازه‌گیری‌ها را حذف می‌کنیم.

منابع و مراجع

[1] Zhao, Jingyi, Gao, Chunhai, and Tang, Tao. A review of sustainable maintenance strategies for single component and multicomponent equipment. *Sustainability*, 14(5):2992, 2022.

[2] Tinga, Tiedo. Application of physical failure models to enable usage and load based maintenance. *Reliability engineering & system safety*, 95(10):1061–1075, 2010.

[3] Wu, Sze-jung, Gebraeel, Nagi, Lawley, Mark A, and Yih, Yuehwern. A neural network integrated decision support system for condition-based optimal predictive maintenance policy. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 37(2):226–236, 2007.

[4] Kaiser, Kevin A and Gebraeel, Nagi Z. Predictive maintenance management using sensor-based degradation models. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, 39(4):840–849, 2009.

[5] Jung, Deokwoo, Zhang, Zhenjie, and Winslett, Marianne. Vibration analysis for iot enabled predictive maintenance. in *2017 ieee 33rd international conference on data engineering (icde)*, pp. 1271–1282. IEEE, 2017.

[6] Carlsson, Anders, Kuzminykh, Ievgeniia, Franksson, Robin, and Liljegren, Alexander. Measuring a lora network: Performance, possibilities and limitations. in *Internet of Things, Smart Spaces, and Next Generation Networks and Systems: 18th International*

Conference, NEW2AN 2018, and 11th Conference, ruSMART 2018, St. Petersburg, Russia, August 27–29, 2018, Proceedings 18, pp. 116–128. Springer, 2018.

[7] Tabbane, Sami. Iot systems overview. *International Telecommunication Union*, 2017.

[8] Ramya, C Muthu, Shanmugaraj, Madasamy, and Prabakaran, R. Study on zigbee technology. in *2011 3rd international conference on electronics computer technology*, vol. 6, pp. 297–301. IEEE, 2011.

[9] Gislason, Drew. *Zigbee wireless networking*. Newnes, 2008.

[10] Song, SM and Yao, WJ. Research on the application value of wireless mesh network in power equipment of the upiot. in *Journal of Physics: Conference Series*, vol. 1346, p. 012046. IOP Publishing, 2019.

[11] Salih, Mohammed A Abdala& Alaa Mohammed. Design and performance analysis of building monitoring system with wireless sensor networks. *Iraqi Journal of Science*, 53(4):1097–1102, 2012.

[12] International, Digi. Xbee zigbee guide. https://www.digi.com/resources/documentation/Digidocs/90001942-13/Default.htm, 2022.

[13] Raspberry pi datasheet. https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf.

[14] Naqvi, Syeda Noor Zehra, Yfantidou, Sofia, and Zimányi, Esteban. Time series databases and influxdb. *Studienarbeit, Université Libre de Bruxelles*, 12, 2017.

[15] Influxdb. https://www.influxdata.com/time-series-platform/, Jun 2023.

[16] Bressert, Eli. Scipy and numpy: an overview for developers. 2012.

[17] Harris, Charles R, Millman, K Jarrod, Van Der Walt, Stéfan J, Gommers, Ralf, Virtanen, Pauli, Cournapeau, David, Wieser, Eric, Taylor, Julian, Berg, Sebastian, Smith, Nathaniel J, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[18] Vue.js. https://vuejs.org/guide/introduction.html.

[19] Influxdb document. https://docs.influxdata.com/influxdb/v2.7/, Jun 2023.

پیوست

موضوعات مرتبط با متن گزارش پایان نامه که در یکی از گروه‌های زیر قرار می‌گیرد، در بخش پیوست‌ها آورده شوند:

۱. اثبات های ریاضی یا عملیات ریاضی طولانی.

۲. داده و اطلاعات نمونه (های) مورد مطالعه (Case Study) چنانچه طولانی باشد.

۳. نتایج کارهای دیگران چنانچه نیاز به تفصیل باشد.

۴. مجموعه تعاریف متغیرها و پارامترها، چنانچه طولانی بوده و در متن به انجام نرسیده باشد.

کد میپل

```
with(DifferentialGeometry):
with(Tensor):
DGsetup([x, y, z], M)
frame name: M
a := evalDG(D_x)
D_x
b := evalDG(-2 y z D_x+2 x D_y/z^3-D_z/z^2)
```

واژه‌نامه‌ی فارسی به انگلیسی

آ

اسکالر . . . . . Scalar

ب

بالابر . . . . . Lift

پ

پایا . . . . . Invariant

ت

تناظر . . . . . Correspondence

ث

ثابت‌ساز . . . . . Stabilizer

ج

جایگشت . . . . . Permutation

چ

چند جمله‌ای . . . . . Polynomial

ح

حاصل‌ضرب دکارتی . . . . Cartesian product

خ

خودریختی . . . . . Automorphism

د

درجه . . . . . Degree

ر

ریزپردازنده . . . . . microprocessor

ز

زیرمدول . . . . . Submodule

س

سرشت . . . . . Character

ص

صادقانه . . . . . Faithful

ض

ضرب داخلی . . . . . Inner product

ط

طوقه . . . . . Loop

ظ

ظرفیت . . . . . Valency

ع

عدم مجاورت . . . . . Nonadjacency

ف

فضای برداری . . . . . Vector space

ک

کاملاً تحویل‌پذیر . . . Complete reducibility

گ

گراف . . . . . Graph

م

ماتریس جایگشتی . . . Permutation matrix

ن

ناهمبند . . . . . Disconnected

و

وارون‌پذیر . . . . . Invertible

ه

همبند . . . . . Connected

ی

یال . . . . . Edge

واژه‌نامه‌ی انگلیسی به فارسی

A

Automorphism . . . . . خودریختی

B

Bijection . . . . . دوسویی

C

Cycle group . . . . . گروه دوری

D

Degree . . . . . درجه

E

Edge . . . . . یال

F

Function . . . . . تابع

G

Group . . . . . گروه

H

Homomorphism . . . . . همریختی

I

Invariant . . . . . پایا

L

Lift . . . . . بالابر

M

Module . . . . . مدول

N

Natural map . . . . . نگاشت طبیعی

O

One to One . . . . . یک به یک

P

Permutation group . . . . . گروه جایگشتی

Q

Quotient graph . . . . . گراف خارج‌قسمتی

R

Reducible . . . . . تحویل پذیر

S

Sequence . . . . . دنباله

T

Trivial character . . . . . سرشت بدیهی

U

Unique . . . . . منحصربفرد

V

Vector space . . . . . فضای برداری

Abstract

This page is accurate translation from Persian abstract into English.

Key Words:

Write a 3 to 5 KeyWords is essential. Example: AUT, M.Sc., Ph. D,..